103. Параметры по умолчанию

На этом уроке мы рассмотрим, что такое параметры по умолчанию в языке C++ и то, как их использовать.

Оглавление:

1. Параметры по умолчанию
2. Несколько параметров по умолчанию
3. Объявление параметров по умолчанию
4. Параметры по умолчанию и перегрузка функций
5. Заключение

Параметры по умолчанию

Параметр по умолчанию (или ***«необязательный параметр»***) — это параметр функции, который имеет определенное (по умолчанию) значение. Если пользователь не передает в функцию значение для параметра, то используется значение по умолчанию. Если же пользователь передает значение, то это значение используется вместо значения по умолчанию. Например:

```
#include <iostream>

void printValues(int a, int b=5)
{
    std::cout << "a: " << a << '\n';
    std::cout << "b: " << b << '\n';
}

int main()
{
    printValues(1); // в качестве b будет использоваться значение по умолчанию - 5
    printValues(6, 7); // в качестве b будет использоваться значение, предоставляемое пользователем - 7
}
```

Результат выполнения программы:

```
a: 1
b: 5
a: 6
b: 7
```

В первом вызове функции мы не передаем аргумент для `b`, поэтому функция использует значение по умолчанию — `5`. Во втором вызове мы передаем значение для `b`, поэтому оно используется вместо параметра по умолчанию.

Параметр по умолчанию — это отличный вариант, когда функция нуждается в значении, которое пользователь может переопределить, а может и не переопределить. Например, вот несколько **прототипов функций**, для которых могут использоваться параметры по умолчанию:

```
void openLogFile(std::string filename="default.log");
int rollDice(int sides=6);
void printStringInColor(std::string str, Color color=COLOR_RED); // Color - это перечисление
```

Несколько параметров по умолчанию

Функция может иметь несколько параметров по умолчанию:

```
void printValues(int a=10, int b=11, int c=12)
{
    std::cout << "Values: " << a << " " << b << " " << c << '\n';
}
```

При следующих вызовах функции:

```
printValues(3, 4, 5);
printValues(3, 4);
printValues(3);
printValues();
```

Результат следующий:

```
Values: 3 4 5
Values: 3 4 12
Values: 3 11 12
Values: 10 11 12
```

Обратите внимание, предоставить аргумент для параметра `c`, не предоставляя при этом аргументы для параметров `a` и `b` — нельзя (перепрыгивать через параметры не разрешается). Это связано с тем, что язык C++ не поддерживает следующий синтаксис вызова функции: `printValues(,,5)`. Из этого вытекают следующие два правила:

Правило №1: **Все параметры по умолчанию в прототипе или в определении функции должны находиться справа.** Следующее вызовет ошибку:

```
void printValue(int a=5, int b); // не разрешается
```

Правильно:

```
void printValue(int a, int b=5);
```

Правило №2: **Если имеется более одного параметра по умолчанию, то самым левым параметром по умолчанию должен быть тот, который с наибольшей вероятностью (среди всех остальных параметров) будет явно переопределен пользователем**.

Объявление параметров по умолчанию

Как только параметр по умолчанию объявлен, повторно объявить его уже нельзя. Это значит, что для функции с предварительным объявлением и определением, параметр по умолчанию объявить можно либо в предварительном объявлении, либо в определении функции, но не в обоих местах сразу. Например:

```
void printValues(int a, int b=15);

void printValues(int a, int b=15) // ошибка: переопределение параметра по умолчанию
{
    std::cout << "a: " << a << '\n';
    std::cout << "b: " << b << '\n';
}
```

Рекомендуется объявлять параметры по умолчанию в предварительном объявлении, а не в определении функции, так как предварительные объявления можно использовать в нескольких файлах — при таком раскладе параметры по умолчанию будет легче увидеть (особенно, если предварительное объявление находится в **заголовочном файле**). Например:

boo.h:

```
#ifndef BOO_H
#define BOO_H
void printValues(int a, int b=15);
#endif
```

main.cpp:

```
#include "boo.h"
#include <iostream>

void printValues(int a, int b)
{
    std::cout << "a: " << a << '\n';
    std::cout << "b: " << b << '\n';
}

int main()
{
    printValues(7);

    return 0;
}
```

Обратите внимание, в примере, приведенном выше, используется параметр по умолчанию `b` для функции printValues(), так как main.cpp подключает boo.h, который имеет предварительное объявление функции printValues() с объявленным параметром по умолчанию.

Правило: Объявляйте параметры по умолчанию в предварительном объявлении функции, в противном случае (если функция не имеет предварительного объявления) — объявляйте в определении функции.

Параметры по умолчанию и перегрузка функций

Функции с параметрами по умолчанию могут быть **перегружены**. Например:

```
void print(std::string string);
void print(char ch=' ');
```

Если пользователь вызовет просто `print()` (без параметров), то выведется пробел, что будет результатом выполнения `print(' ')`.

Однако, стоит отметить, что параметры по умолчанию НЕ относятся к параметрам, которые учитываются при определении уникальности функции. Следовательно, следующее не допускается:

```
void printValues(int a);
void printValues(int a, int b=15);
```

При вызове `printValues(10)` компилятор не сможет определить, хотите ли вы вызвать `printValues(int)` или `printValues(int, 15)` (со значением по умолчанию).

Заключение

Параметры по умолчанию — это полезный механизм указания параметров, при котором пользователь может переопределять значения по умолчанию, либо не переопределять их вообще. Они часто используются в языке C++, и их применение вы увидите уже на следующих уроках.